Language model

Từ bigram đến cách sinh một chuỗi

Phase 6 · Bài 6.4

Dự đoán token tiếp theo

Sau “em học”: “toán”, “tin”, hoặc một token không hợp ngữ cảnh.

Model cho phân phối, ta chọn một token rồi lặp.

Next-token prediction khác phân loại cả câu.

Trôi chảy chưa chứng minh đúng sự thật.

Mục tiêu buổi này

- Tính bigram và smoothing.
- Dựng label dịch một vị trí; tính NLL và perplexity.
- So greedy, sampling, top-k, top-p và beam search.
- Đọc giới hạn điểm language model.

01

Xác suất của chuỗi

Quy tắc nhân xác suất

BOS đánh dấu bắt đầu, EOS đánh dấu kết thúc.

Bigram: một token trước. Causal transformer: cả tiền tố.

N-gram: ngữ cảnh, phần lớn không bao giờ được đếm.

HỎI NHANH

Một model không dùng EOS, nên xác suất của một câu là tích xác suất của các token chữ. So câu hoàn chỉnh “em học” với câu “em học toán”, điều gì luôn đúng?

A “em học toán” cao hơn, vì có thêm một từ hợp ngữ cảnh

B “em học” không thấp hơn, vì nó là tiền tố của câu kia

C Hai câu bằng nhau, vì có chung tiền tố “em học”

D Không so được khi chưa biết

Corpus nhỏ có thể đếm tay

“em học toán”, “em học tin”, “em đọc sách”, “bạn học toán”, “bạn đọc sách”.

KẾT QUẢ

```
vocabulary size: 9
sau 'em': {'học': 2, 'đọc': 1}
```

9 token đích: 7 từ, EOS và UNK; BOS chỉ làm điều kiện đầu vào.

Bảng đếm bigram

	<eos>	bạn	em	học	sách	tin	toán	đọc
<bos>		2	3					
em				2				1
bạn				1				1
học						1	2	
đọc					2			
toán	2							

ô trống: chưa gặp, xác suất 0 nếu không làm trơn

Số đếm bigram trên năm câu train của bài (hàng: token trước, cột: token sau): hầu hết các ô bằng 0, nên một cặp chưa gặp có xác suất 0 nếu không làm trơn.

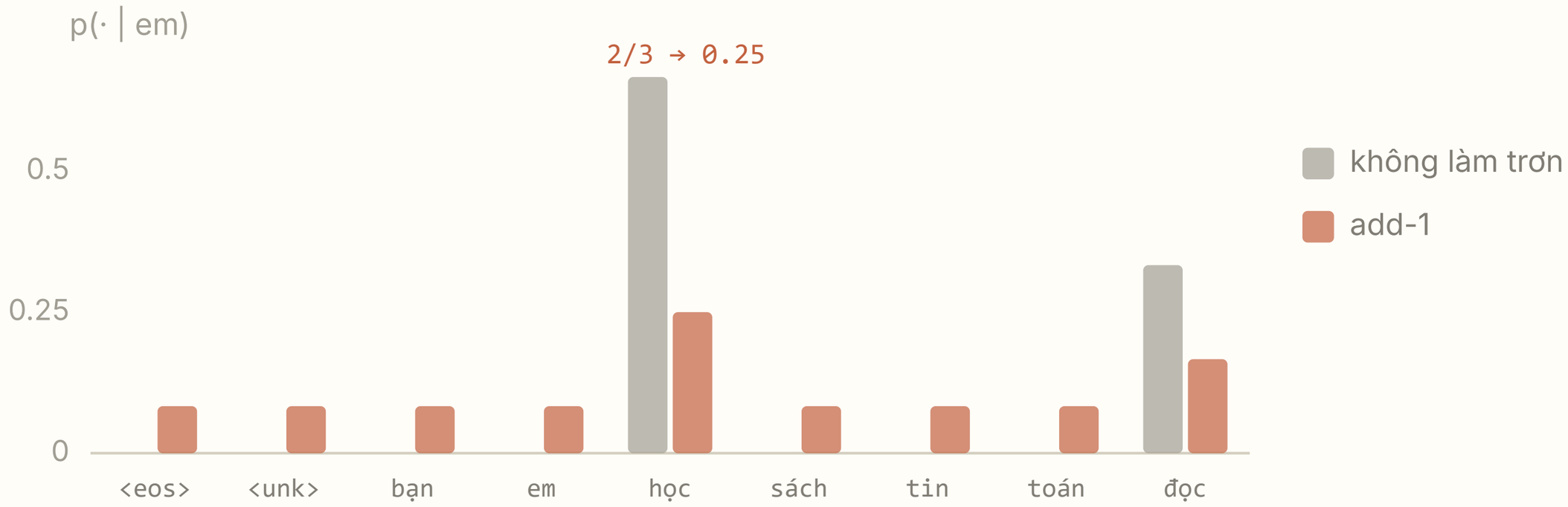
Add-α smoothing

:

Không làm trơn: . Cặp chưa thấy có xác suất 0.

Vocabulary lớn: số đếm giả lấn át số đếm thật.

Phân phối sau “em”



Phân phối token sau “em” (số từ code bài): không làm trơn, “học” chiếm 2/3 và bảy token còn lại bằng 0; add-1 hạ “học” xuống 0.25 và cho mọi token một phần xác suất.

HỎI NHANH

Bỏ smoothing, tức . Một câu validation có 20 cặp token. 19 cặp đã gặp trong train với xác suất cao, còn một cặp chưa gặp. NLL trung bình của câu bằng bao nhiêu?

A Gần NLL của 19 cặp kia, vì một cặp chỉ chiếm 1/20

B Bằng NLL của 19 cặp kia, vì cặp chưa gặp bị bỏ qua

C Vô cùng, vì không hữu hạn

D Lớn nhưng hữu hạn, vì xác suất ấy chỉ rất nhỏ

HỎI NHANH

Giữ đúng số đếm sau “em”, gồm “học” 2 lần và “đọc” 1 lần, nhưng vocabulary đích có 10 000 token. Với add-1, xấp xỉ bao nhiêu?

A 0.0003

B 0.25, như với vocabulary 9 token

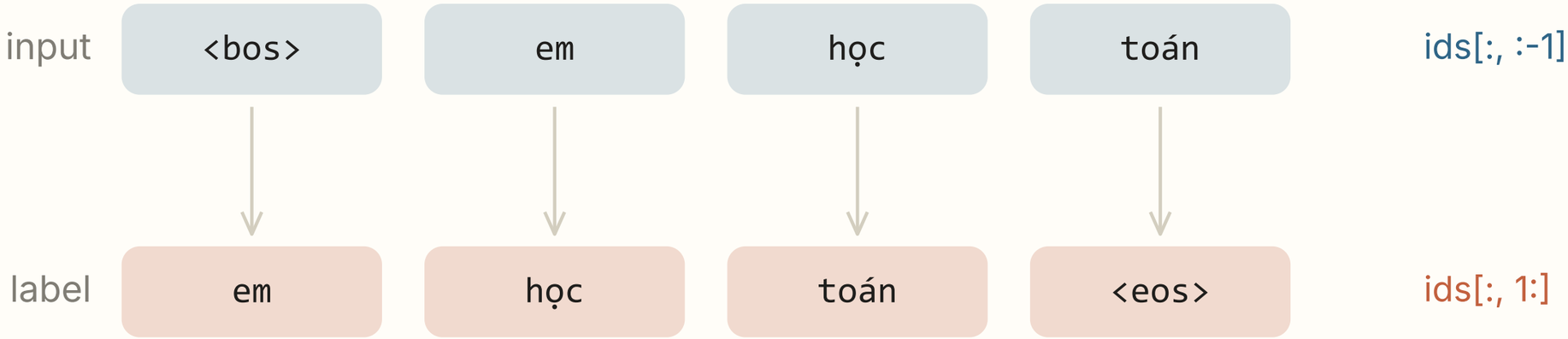
C 0.6, vì chỉ token đã gặp mới được cộng 1

D 0.667, vì add-1 giữ nguyên tỉ lệ 2:1

02

Học và chấm đúng vị trí

Input và label dịch một vị trí



vị trí i chỉ đọc tiền tố đến i, dự đoán token i + 1

Next-token prediction trên chuỗi của bài: input bỏ token cuối, label bỏ token đầu, nên vị trí i đọc tiền tố đến i và dự đoán token i + 1.

Cross-entropy và perplexity

là số token đích hợp lệ. Xác suất đúng ở mọi bước cho PPL bằng 4.

Các probe cho bigram

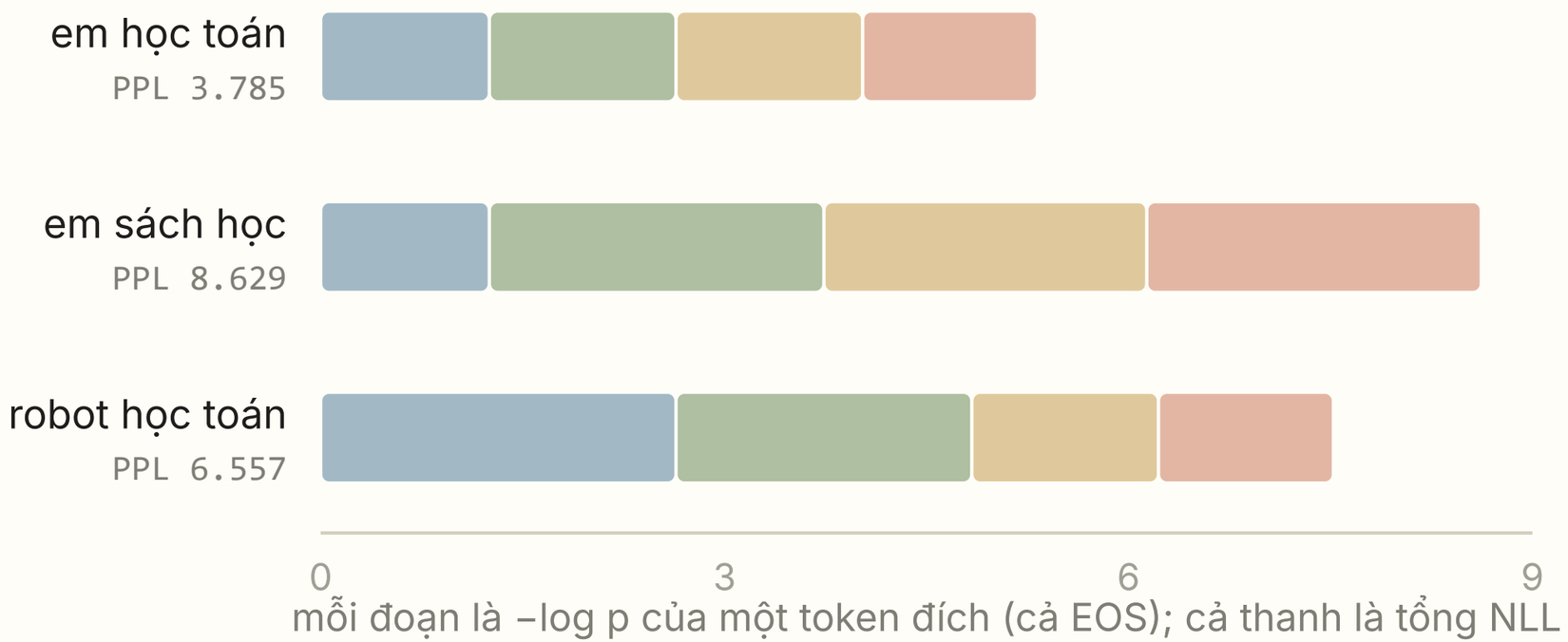
Câu	NLL	PPL
em học toán	1.331	3.785
em sách học	2.155	8.629
robot học toán	1.880	6.557

Mỗi câu có 4 token đích kể EOS.

“robot” bị chuyển thành UNK.

Đây là probe, chưa phải corpus evaluation độc lập.

NLL từng token



NLL từng token của ba câu probe dưới bigram add-1 (số từ code bài): “em học toán” có PPL 3.785; đảo trật tự thành “em sách học” lên 8.629; từ lạ “robot” bị đổi thành UNK cho 6.557.

Bigram viết như causal LM

```
inputs, labels = sequence[:-1], sequence[1:]
logits = bigram_logits(inputs)
F.cross_entropy(logits, label_ids)
```

KẾT QUẢ

```
logits shape: (4, 9)
cross-entropy: 1.331
thêm 2 vị trí đệm: 1.331
```

Logits là log-xác suất của bigram.

Loss bằng NLL của “em học toán”.

Label `-100` không vào loss.

HỎI NHANH

Batch `ids` có shape `(8, 128)`. Model trả về `logits` shape `(8, 128, V)`, trong đó logits ở vị trí dự đoán token . Đưa cặp nào vào cross-entropy mới đúng?

A `logits[:, :-1]` với `ids[:, 1:]`

B `logits[:, 1:]` với `ids[:, :-1]`

C `logits[:, :-1]` với `ids[:, :-1]`

D `logits` với `ids`, không cắt gì

HỎI NHANH

Hai câu dùng để đánh giá có 2 và 8 token hợp lệ. Muốn NLL trung bình theo token thì gộp thế nào?

A Lấy trung bình hai loss câu, mỗi câu trọng số 1/2

B Bỏ câu ngắn, vì 2 token quá ít để đo

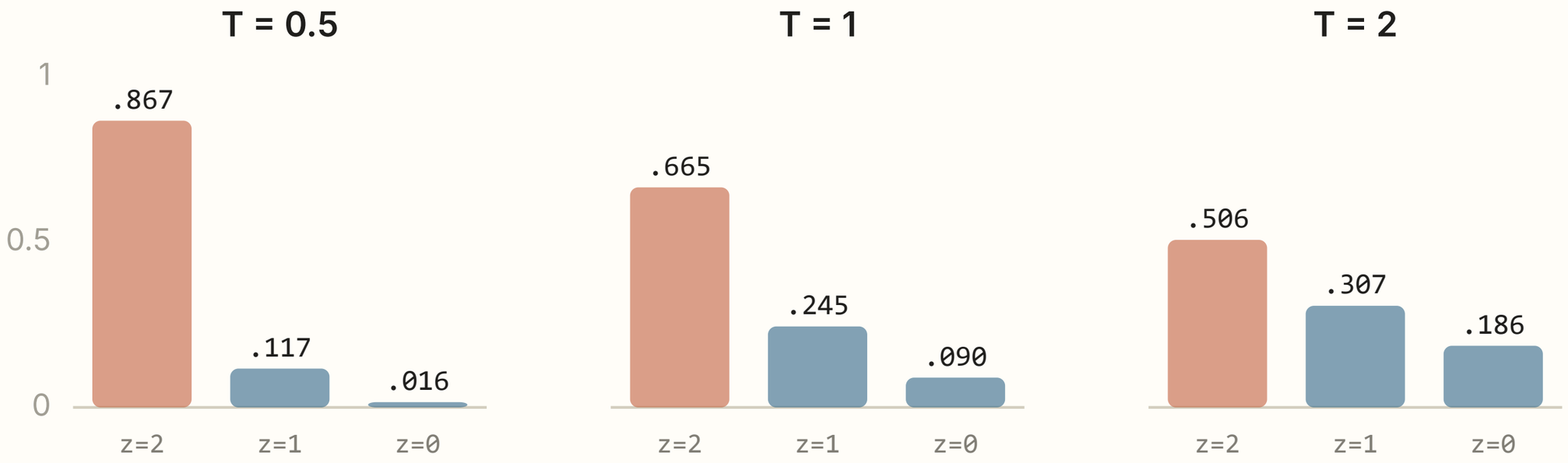
C Cộng NLL của cả 10 token rồi chia cho 2 câu

D Cộng NLL của cả 10 token rồi chia cho 10

03

Chọn token từ phân phối

Temperature đổi độ nhọn



thứ tự không đổi; T nhỏ làm nhọn, T lớn làm phẳng

Cùng logits (2, 1, 0) qua softmax với ba temperature (số từ code bài): T = 0.5 cho 0.867 ở token đầu, T = 2 hạ xuống 0.506; thứ tự xác suất giữ nguyên.

Greedy và sampling

Greedy: lấy argmax mỗi bước.

Sampling: rút theo phân phối.

Temperature dương không đổi argmax.

: gần greedy. : gần đều.

Công thức không dùng được với . “Temperature 0” trong API là quy ước greedy.

Seed không chứng minh chất lượng.

Top-k và top-p

Top-k giữ token cao nhất.

Top-p giữ tiền tố nhỏ nhất đạt tổng xác suất .

Temperature áp trước, rồi mới lọc.

KẾT QUẢ

```
base: [0.665 0.245 0.09 ]
top-p=0.8: [0.731 0.269 0.   ]
số token giữ: 2
```

Giữ cả token làm vượt ngưỡng.

HỎI NHANH

Vẫn logits (2, 1, 0), nhưng hạ temperature từ 1 xuống 0.5 rồi mới lọc. Top-p 0.8 và top-k 2 lần lượt giữ bao nhiêu token?

A Top-p giữ 2, top-k giữ 2

B Top-p giữ 1, top-k giữ 2

C Top-p giữ 1, top-k giữ 1

D Top-p giữ 2, top-k giữ 1

HỎI NHANH

Hai bước sinh có phân phối đã sắp là (0.97, 0.01, 0.01, 0.01) và (0.3, 0.3, 0.2, 0.2). Top-p 0.9 giữ bao nhiêu token ở mỗi bước?

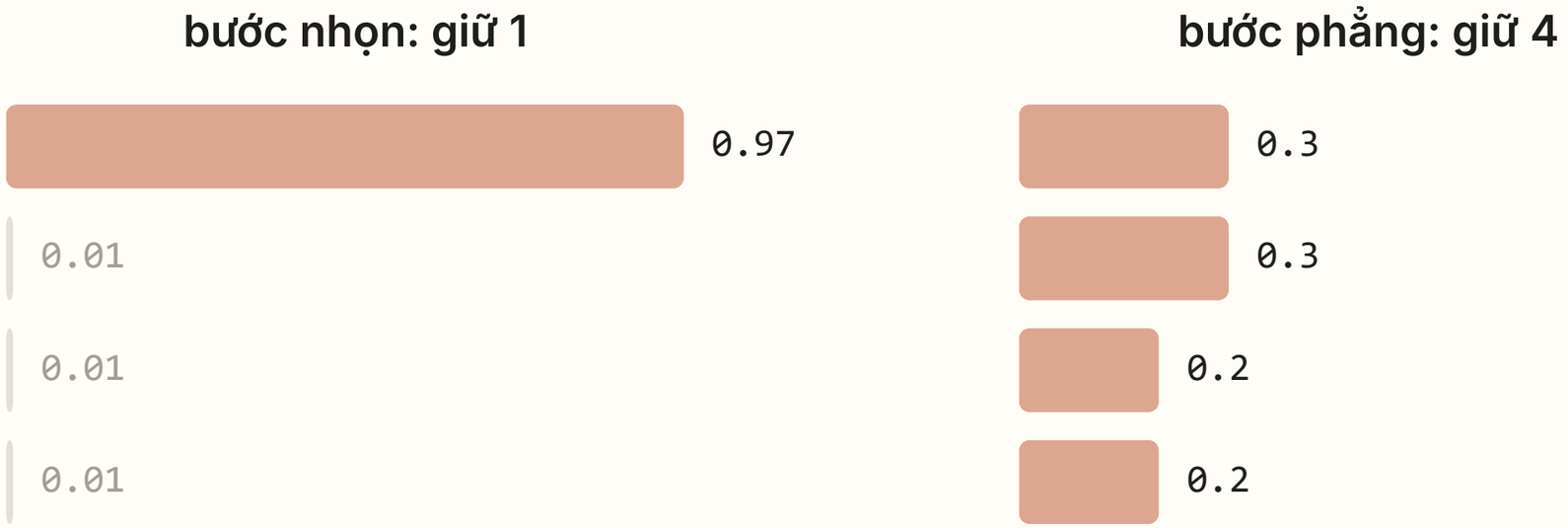
A 2 và 2, như top-k 2

B 4 và 1

C 1 và 4

D 1 và 3

Top-p co giãn theo độ nhọn



top-p 0.9 giữ token cho tới khi tổng dồn đạt 0.9, kể cả token vượt ngưỡng

Top-p 0.9 trên hai phân phối của câu hỏi: bước nhọn chỉ cần giữ 1 token (0.97), bước phẳng phải giữ cả 4 token mới đạt tổng 0.9.

Vòng sinh bigram

Dừng ở EOS hoặc sau tối đa 6 token.

KẾT QUẢ

```
greedy: em học toán
sampling seed 0: em em
```

Greedy đi đường được đếm nhiều nhất.

“em em” chưa gặp, nhưng smoothing cho xác suất 1/12.

Giới hạn sinh để dừng, không sửa chất lượng.

HỎI NHANH

Bỏ giới hạn số token. Sinh greedy trên một bigram có 9 token đích, trong đó có EOS. Đã sinh 9 token mà chưa gặp EOS. Điều gì là chắc chắn?

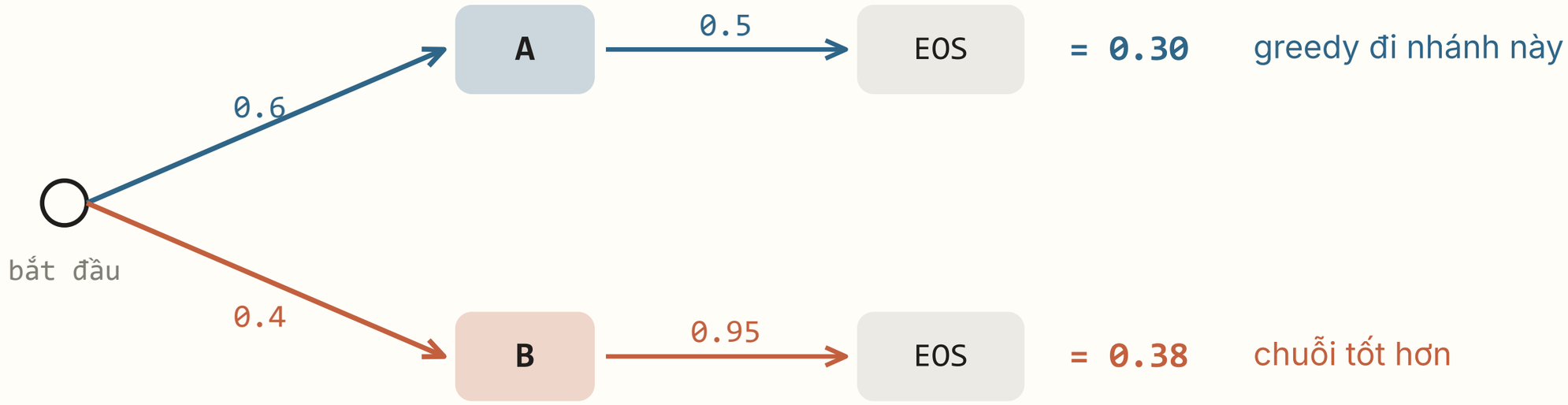
A Sắp gặp EOS, vì smoothing cho EOS xác suất dương

B Chưa kết luận được, phải chạy tiếp mới biết

C Chỉ lặp mãi nếu có hai token liền nhau giống nhau

D Sẽ không bao giờ gặp EOS, vì chuỗi đã vào một chu trình

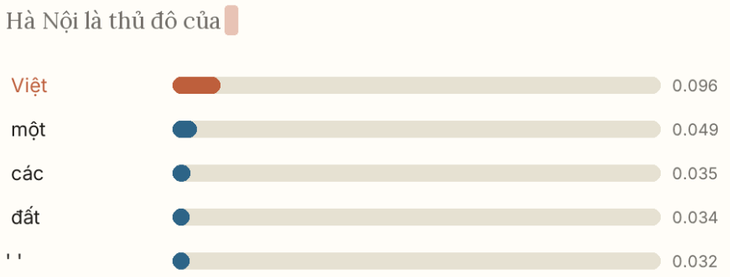
Greedy nhìn từng bước, beam giữ nhiều đường



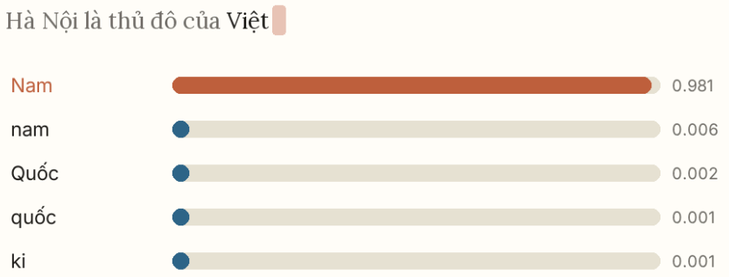
beam 2 giữ cả A lẫn B nên tìm ra B EOS

Greedy chọn A (0.6) ở bước đầu nên được A EOS với xác suất 0.30; chuỗi B EOS có 0.4 · 0.95 = 0.38 cao hơn, và beam độ rộng 2 giữ cả hai nhánh nên tìm ra nó.

Một language model thật sinh từng token



bước 1: 5 token có xác suất cao nhất; greedy chọn “Việt” (0.10)



bước 2: 5 token có xác suất cao nhất; greedy chọn “Nam” (0.98)



bước 6: 5 token có xác suất cao nhất; greedy chọn “có” (0.13)

Hà Nội là thủ đô của Việt Nam, là nơi có nhiều người dân

sau 9 bước greedy

Qwen2.5-0.5B viết tiếp “Hà Nội là thủ đô của” từng token: mỗi bước model trả một phân phối, greedy lấy token cao nhất rồi nối vào.

Greedy chưa tối ưu cả chuỗi

A EOS: .

B EOS: .

Greedy chọn A ở bước đầu nên bỏ lỡ B EOS.

Beam search độ rộng

- Giữ tiền tố có tổng log cao nhất.
- Mỗi bước: ứng viên, giữ lại .
- là greedy; tìm ra B EOS.
- Tiền tố bị loại không quay lại: vẫn gần đúng.
- Length penalty đổi mục tiêu xếp hạng.

HỎI NHANH

Giữ , và như hình, nhưng đổi thành . Chỉ xét hai chuỗi `A EOS` và `B EOS`. Khi nào chuỗi greedy trả về không kém chuỗi còn lại?

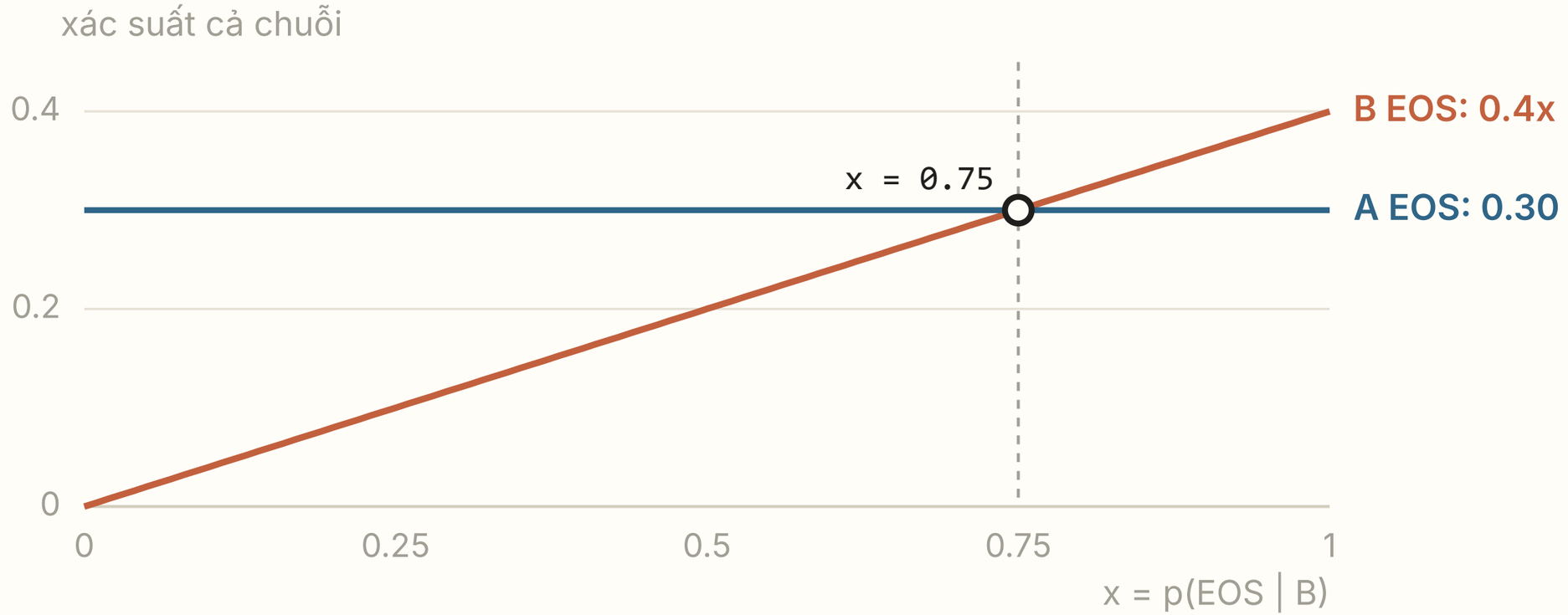
A Với mọi , vì A thắng ở bước đầu

B Khi , tức không vượt

C Khi

D Khi , tức nhỏ hơn

Ngưỡng x = 0.75



Greedy luôn trả A EOS với xác suất 0.30; chuỗi B EOS có 0.4x, nên greedy không kém chỉ khi x ≤ 0.75.

Văn bản bị máy sửa

- PPL thấp không xác định ai viết.
- Tác giả, chủ đề, dịch và sửa chính tả là yếu tố gây nhiễu.
- Split theo văn bản nguồn; bản gốc và bản sửa cùng nhóm.
- Báo false positive và false negative trên cách sửa mới.

HỎI NHANH

Một detector gắn nhãn “máy viết” khi PPL dưới một ngưỡng. Ngưỡng được chọn trên validation gồm bài văn của học sinh chuyên Văn và văn bản do model sinh. Đem detector chấm bài tiếng Anh của học sinh mới học, câu ngắn, từ thông dụng. Rủi ro lớn nhất là gì?

A False negative tăng, vì câu ngắn có PPL cao

B False positive tăng, vì câu đơn giản có PPL thấp

C Không có gì thay đổi, vì ngưỡng đã được chọn trên validation

D Không có gì thay đổi, vì PPL không phụ thuộc người viết

Khi đi thi

- In một cặp input và label trước khi train.
- Đổi token cuối: logits các vị trí trước phải giữ.
- Validation PPL gần 1 quá sớm: nghi lộ tương lai.
- PPL chỉ so khi cùng tokenizer, vocabulary, UNK.
- Chốt cấu hình sinh và dự toán thời gian trên đúng máy.

04

Luyện đề

LUYỆN ĐỀ 1

Một đội train một causal LM nhỏ. Sau 300 bước, validation perplexity đã là 1.03, nhưng sinh greedy thì ra chuỗi vô nghĩa. Label đã được dịch đúng một vị trí. Nên kiểm gì trước?

A Tăng temperature, vì greedy làm chuỗi lặp lại

B Train thêm, vì perplexity chưa về đúng 1

C Kiểm causal mask, vì PPL gần 1 gợi ý tương lai bị lộ

D Đổi tokenizer, vì perplexity phụ thuộc tokenizer

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 231

LUYỆN ĐỀ 2

Cấu hình sinh là `do_sample=True, top_k=1, temperature=1.5`. Chạy với 5 seed khác nhau, cả 5 output trùng nhau đến từng token. Vì sao?

A Vì temperature 1.5 quá cao nên sampling bị tắt

B Vì seed không được truyền vào bộ sinh nên RNG không đổi

C Vì top-k 1 giữ một token ngẫu nhiên, cố định theo prompt

D Vì top-k 1 chỉ giữ argmax, nên lần nào cũng là greedy

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 241

LUYỆN ĐỀ 3

Mọi câu có đúng hai token. Ở bước đầu, , , . Token thứ hai tốt nhất sau mỗi nhánh có xác suất có điều kiện 0.6 sau A, 0.5 sau B và 1.0 sau C. Beam search độ rộng 2 trả về câu có xác suất bao nhiêu?

A 0.24, vì câu tốt nhất đã bị loại ngay ở bước đầu

B 0.24, và đó chắc chắn là câu tối ưu toàn cục

C 0.25, vì beam search xét đủ mọi nhánh

D 0.175, vì beam search bỏ nhánh mà greedy chọn

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 18

Chốt ý

Dự đoán token tiếp theo cho xác suất cả chuỗi; EOS kết thúc câu.

NLL và PPL chỉ so được khi cùng đơn vị và mẫu số.

Decoding và tính đúng là hai vấn đề khác nhau.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi temperature và top-p với cùng seed, ghi lại cấu hình rồi so các câu sinh ra.